

Explanation of the Uniform Cost Search Algorithm

1 Purpose of the Algorithm

The method `computeCostFromAtoB(Graph graph, byte initialStateId, byte finalStateId)` computes the minimum path cost from a start node to a goal node in a weighted graph.

This implementation is based on **Uniform Cost Search (UCS)**, which always expands the node with the currently lowest cumulative cost. In graphs with non-negative edge weights, this guarantees that the first time the goal node is removed from the priority queue, its cost is optimal.

2 Main Idea

The algorithm keeps track of:

- a **priority queue** called `frontier`, which stores nodes that still need to be explored, ordered by cost;
- an array called `bestCost`, which stores the cheapest known cost to reach every node.

The process begins at the initial node with cost 0. Then:

1. the cheapest node is extracted from the priority queue;
2. if it is the goal, the algorithm returns its cost;
3. otherwise, all of its neighbors are examined;
4. whenever a cheaper path to a neighbor is found, that neighbor is updated and inserted into the queue.

3 Step-by-Step Explanation

3.1 Initial Check

```
if (graph.getAdjacencyListOf(initialStateId).isEmpty()) return -2;
```

If the initial node has no outgoing edges, the algorithm immediately returns -2. This is treated as a special case meaning that the search cannot even begin from the given start node.

3.2 Initialization of the Frontier

```
PriorityQueue<SearchNode> frontier = initializePriorityQueue(  
    initialStateId);
```

The frontier is a priority queue containing nodes waiting to be explored. It starts with only the initial node, whose cost is 0.

```
private static PriorityQueue<SearchNode> initializePriorityQueue(  
    byte initialStateId) {  
    PriorityQueue<SearchNode> pq = new PriorityQueue<SearchNode>()  
        ;  
    pq.add(new SearchNode(initialStateId, (short) 0));  
    return pq;  
}
```

3.3 Initialization of Best Costs

```
short[] bestCost = initializeBestCost(graph.getSize(),  
    initialStateId);
```

This array stores the best known cost to each node. Initially:

- every node receives `Short.MAX_VALUE`, meaning “infinity” or “not yet reached”;
- the initial node receives cost 0.

```
private static short[] initializeBestCost(byte graphSize, byte  
    initialStateId) {  
    short[] bc = new short[graphSize];  
    Arrays.fill(bc, Short.MAX_VALUE);  
    bc[initialStateId] = 0;  
    return bc;  
}
```

3.4 Main Search Loop

```
while(!frontier.isEmpty()) {
```

The algorithm continues as long as there are nodes left to explore.

3.5 Extract Cheapest Node

```
SearchNode currentNode = frontier.poll();  
byte currentNodeId = currentNode.getId();  
short currentCost = currentNode.getCost();
```

The priority queue always removes the node with the smallest cumulative cost. This is the core principle of Uniform Cost Search.

3.6 Ignore Outdated Entries

```
if (currentCost > bestCost[currentNodeId])
    continue;
```

A node may appear multiple times in the priority queue if a better path to it is found later. If the popped version has a higher cost than the one currently recorded in `bestCost`, it is outdated and skipped.

3.7 Goal Test

```
if (currentNodeId == finalStateId)
    return currentCost;
```

If the node taken from the queue is the goal node, the algorithm returns its cost immediately. Because UCS expands nodes in increasing cost order, this returned value is the minimum possible path cost.

3.8 Skip Dead Ends

```
if (currentAdjacencyList.isEmpty())
    continue;
```

If the current node has no neighbors, there is nothing more to explore from it.

3.9 Relaxation of Edges

```
for (Edge edge : currentAdjacencyList) {
    byte neighbourId = edge.getDestinationId();
    short newCost = (short) (edge.getWeight() + currentCost);

    if (newCost < bestCost[neighbourId]) {
        bestCost[neighbourId] = newCost;
        frontier.add(new SearchNode(neighbourId, newCost));
    }
}
```

For every outgoing edge:

- the algorithm computes the new cumulative cost to reach the neighbor;
- if this new cost is smaller than the currently known one, the neighbor is updated;
- the neighbor is then inserted into the priority queue for future exploration.

This operation is equivalent to the standard **relaxation step** used in shortest-path algorithms.

4 Return Values

The method may return three kinds of values:

- -2: the initial node has no neighbors, so the search is not started;
- -1: no path to the goal exists;
- a non-negative number: the minimum cost from the initial node to the final node.

5 Relation to Dijkstra's Algorithm

This implementation of Uniform Cost Search is essentially equivalent to **Dijkstra's shortest-path algorithm** for a single destination:

- both use a priority queue;
- both expand the least-cost node first;
- both maintain best-known path costs;
- both assume non-negative edge weights.

The main difference is interpretational:

- **Uniform Cost Search** is usually described in the context of search problems;
- **Dijkstra's algorithm** is usually described in the context of graph shortest paths.

6 Time Complexity

Let:

- V be the number of vertices;
- E be the number of edges.

Using a priority queue, the time complexity is approximately:

$$O((V + E) \log V)$$

In practice, since nodes can be inserted multiple times into the queue, the implementation remains efficient but may process outdated entries, which are filtered by the condition:

```
if (currentCost > bestCost[currentNodeId]) continue;
```

7 Summary

This algorithm finds the cheapest path from a start node to a goal node by:

1. always exploring the lowest-cost available node first;
2. storing the best known cost to each node;
3. ignoring outdated queue entries;
4. stopping as soon as the goal is reached optimally.

Therefore, it is a correct and efficient implementation of Uniform Cost Search for weighted graphs with non-negative edge costs.